BACKEND ENGINEERING

DAY 3 & DAY 4

Validation • Error Handling • Project Structure • Database Integration

Practical + Research-Oriented Learning

Learn → Understand → Set Up → Explore → Build → Test → Explain

Your starting point: You have completed the first two lessons: backend basics, HTTP, FastAPI routes, path parameters, query parameters, request bodies and introductory Pydantic models. These notes continue from that level and assume that database integration is new.

Today's direction: First make incoming data safer and API errors clearer. Then organise the application into modules and connect it to a persistent database. MongoDB is the main path; PostgreSQL is included as the relational alternative.

Learning rule: do not rush to copy code. For every function, explain what enters, what happens, what comes out, where data is stored, and what can fail.

Suggested time: Day 3 — 2.5–3 hours | Day 4 — 3.5–4.5 hours

How to use these two lessons

Follow the same learning rhythm as Days 1 and 2. Read a small concept, inspect the example, predict its behaviour, run it, change one thing, and explain the result in your own words. Use the Hospital ERP example throughout.

Lesson	Main focus	Outcome
Day 3	Pydantic validation, optional fields, response models, HTTPException and useful status codes	Accept valid data, reject invalid data deliberately, and return clear API responses.
Day 4 — Part A	Project structure, modules, APIRouter, dependencies and application lifespan	Split a growing FastAPI application into understandable files.
Day 4 — Part B	Database concepts and MongoDB connection with PyMongo Async	Create, read, update and delete persistent records through FastAPI.
Day 4 — Alternative	PostgreSQL with SQLModel	Understand the relational option and how it differs from MongoDB.

Before you begin

• Use the same VS Code project and virtual environment from Day 1 if it is still working.

• Keep your Day 1 and Day 2 examples as reference files; do not overwrite them.

• Use fake patient data only. Do not place real patient information in a learning database.

• Database connection strings and passwords belong in environment variables, not in source code or Git.

Important terminology: you wrote “prequel database”. I have interpreted this as PostgreSQL, the relational database. If you meant a different database, the lesson can be adjusted.

DAY 3 — Pydantic Validation and Error Handling

Today's goal: make your API check incoming data carefully, distinguish create and update rules, and return useful HTTP responses when something goes wrong.

1. What you will learn today

• Why accepting any input is dangerous.

• How Pydantic Field adds rules such as minimum length, numeric ranges and patterns.

• How to use separate create, update and response models.

• How optional fields and partial updates work.

• How HTTPException communicates expected failures.

• How response_model controls the shape of returned data.

• How to test valid and invalid inputs through /docs.

2. Connect Day 2 to Day 3

In Day 2, PatientCreate described the expected fields and basic Python types. For example, age: int rejected text that could not be interpreted as an integer. Day 3 adds business-friendly constraints: a name should not be blank, age should be within a sensible range, and a response should not expose private fields.

Day 2 concept	Day 3 extension
name: str	name: str = Field(min_length=2, max_length=80)
age: int	age: int = Field(ge=0, le=120)
One model for input	Different models for create, update and output
Return a dictionary	Use response_model to define the public response
Validation error	Handle missing records and business-rule failures with HTTPException

3. Pydantic Field — add rules to your data

Pydantic checks that incoming values match the model. Field lets you describe constraints beyond the basic type. FastAPI uses the model to validate a request body before your route function runs.

```
from pydantic import BaseModel, Field

class PatientCreate(BaseModel):
    name: str = Field(min_length=2, max_length=80)
    age: int = Field(ge=0, le=120)
    department: str = Field(min_length=2, max_length=50)
    email: str | None = None
```

Read the rules: min_length=2 prevents a one-character or empty name; max_length limits overly long input; ge means greater than or equal to; le means less than or equal to. email is optional because its default is None.

Validation is not the same as business approval. A valid age may still be unsuitable for a particular workflow. Put business rules in a service or route-level decision, not only in field types.

4. Create model, update model and response model

One model should not automatically be used for every job. Creating a patient may require name, age and department. Updating a patient may allow only some fields. Returning a patient should expose only fields that clients are allowed to see.

```
from pydantic import BaseModel, Field

class PatientCreate(BaseModel):
    name: str = Field(min_length=2, max_length=80)
```

```
    age: int = Field(ge=0, le=120)
    department: str = Field(min_length=2, max_length=50)

class PatientUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=80)
    age: int | None = Field(default=None, ge=0, le=120)
    department: str | None = Field(default=None, min_length=2, max_length=50)

class PatientPublic(BaseModel):
    id: str
    name: str
    age: int
    department: str
```

The update model uses defaults so that a client can send only the fields it wants to change. Later, use model_dump(exclude_unset=True) to distinguish fields the client actually supplied from fields it left out.

5. HTTPException — return a useful error

Validation checks the shape and values of input. Your application must also handle situations such as a patient ID that does not exist. FastAPI's HTTPException lets a route stop processing and return an HTTP error response.

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

fake_patients = {
    1: {"id": "1", "name": "Ravi", "age": 35, "department": "Cardiology"}
}

@app.get("/patients/{patient_id}")
def get_patient(patient_id: int):
    patient = fake_patients.get(patient_id)

    if patient is None:
        raise HTTPException(
            status_code=404,
            detail="Patient not found"
        )

    return patient
```

Try /patients/1 and /patients/99 in /docs. The first returns a record; the second returns 404. The route deliberately raises an error instead of returning a misleading success message.

Status	Meaning	Hospital ERP example
200 OK	Request succeeded	Patient details returned
201 Created	A new resource was created	Patient registration succeeded
204 No Content	Succeeded with no response body	Deletion succeeded
400 Bad Request	Request cannot be processed as sent	Invalid operation according to API rules
401 Unauthorized	Authentication is missing or invalid	No valid login credentials
403 Forbidden	User is known but not allowed	Reception user attempts an admin-only action
404 Not Found	Requested resource does not exist	Patient ID is not in the database
409 Conflict	Request conflicts with existing state	Duplicate unique registration number
422 Unprocessable Content	Input failed validation	Age is outside the allowed range
500 Internal Server Error	Unexpected server-side failure	Unhandled bug or database failure

6. Response models — control what leaves your API

A response_model tells FastAPI the public shape of a successful response. It helps document, validate and filter output. This matters when database documents contain internal fields that should not be sent to clients.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class PatientPublic(BaseModel):
    id: str
    name: str
    department: str

@app.get("/patients/{patient_id}", response_model=PatientPublic)
def get_patient(patient_id: int):
    patient = {
        "id": str(patient_id),
        "name": "Ravi",
        "department": "Cardiology",
        "internal_note": "not for public response"
    }
    return patient
```

The response model describes the intended public response. In a real application, also enforce authentication and authorization before exposing records.

7. Practical lab — test success and failure

1. Create a new file day3_validation.py or use a separate Day 3 practice folder.

2. Add PatientCreate with the Field constraints shown above.

3. Create POST /patients that accepts PatientCreate and returns the validated data.

4. Add GET /patients/{patient_id}; return 404 when the record is missing.

5. Add a response model that contains only id, name and department.

6. Open /docs and test a valid request, a missing field, a one-character name, age=-5, age=150 and an unknown patient ID.

7. Record the status code and response body for every test.

8. Common beginner mistakes

Mistake	Better habit
Assuming type hints enforce every business rule	Use Field constraints and explicit business checks.
Returning 'not found' with status 200	Raise HTTPException with status 404.
Using one model for all operations	Separate create, update and public response models.
Updating every field even when omitted	Use model_dump(exclude_unset=True) for partial updates.
Returning internal database fields	Define and use a public response model.
Catching every exception and returning success	Handle expected errors clearly; log unexpected failures and return a safe 500 response.

9. Day 3 knowledge check

■ What does Pydantic validate, and what does it not decide?
Answer: _____

■ What do min_length, max_length, ge and le mean?
Answer: _____

■ Why might PatientCreate and PatientPublic be different models?
Answer: _____

■ What is the difference between an omitted update field and a field explicitly set to null?
Answer: _____

■ When should an API return 404?
Answer: _____

■ What is the purpose of response_model?
Answer: _____

■ What is the difference between 401 and 403?
Answer: _____

■ Why should a database error not be returned as a successful response?
Answer: _____

10. Day 3 assignment

• Build POST /doctors with name, experience_years and branch validation.

• Reject negative experience and blank names.

• Build GET /doctors/{doctor_id}; return 404 for an unknown ID.

• Create a public response model that does not include an internal_note field.

• Test at least five invalid requests in /docs and write down the status codes.

Day 3 completion checklist

• I can add validation rules with Pydantic Field.

• I can explain create, update and response models.

• I can use HTTPException for expected errors.

• I understand 201, 404, 409 and 422 in practical situations.

• I can test valid and invalid input in /docs.

• I can explain why a response model matters.

DAY 4 — Organising FastAPI and Connecting a Database

Today's goal: move beyond a single main.py file, understand how routers and shared resources fit together, and connect your API to persistent storage. MongoDB is the main hands-on database in this lesson.

PART A — Organise a growing FastAPI project

1. Why split one file into multiple files?

In Day 1 and Day 2, keeping examples in main.py made the concepts easy to see. As the API grows, putting every route, model, database connection and business rule in one file becomes difficult to maintain. Python modules let you group related responsibilities.

File / folder	Responsibility
app/main.py	Creates the FastAPI app and includes routers.
app/routers/patients.py	Patient endpoints: URL paths and request/response handling.
app/models/patient.py	Pydantic models describing input and output.
app/database.py	Database client and database reference.
app/config.py	Reads configuration from environment variables.
app/services/patient_service.py	Business operations, kept separate as the project grows.

2. Recommended learning project structure

```
backend-day-04/
|
+-- .venv/
+-- .env
+-- .gitignore
+-- requirements.txt
|
+-- app/
    +-- __init__.py
    +-- main.py
    +-- config.py
    +-- database.py
    |
    +-- models/
    |   +-- __init__.py
    |   +-- patient.py
    |
    +-- routers/
        +-- __init__.py
        +-- patients.py
```

The __init__.py files mark these folders as Python packages and make imports predictable. Keep the first project small; add service and repository layers when the application needs them.

3. APIRouter — routes grouped by feature

APIRouter works like a smaller route group. A patient router can own patient URLs, while a doctor router owns doctor URLs. The main app includes each router.

```python
# app/routers/patients.py
from fastapi import APIRouter

router = APIRouter(prefix="/patients", tags=["Patients"])

@router.get("/")
async def list_patients():
    return {"patients": []}

@router.get("/{patient_id}")
async def get_patient(patient_id: str):
    return {"patient_id": patient_id}
```

```
# app/main.py
from fastapi import FastAPI
from app.routers import patients

app = FastAPI(title="Hospital ERP API")

app.include_router(patients.router)

@app.get("/")
async def home():
    return {"message": "Hospital ERP API is running"}
```

The prefix means the route paths become /patients/ and /patients/{patient_id}. tags groups these operations in /docs. include_router() registers the router's operations with the main app.

4. What is a dependency?

A dependency is a reusable function or resource that FastAPI can provide to an endpoint. A database session/client, current user or shared permission check can be provided this way. For this first database exercise, the app lifespan will create one database client and the router will access the selected database.

Architect mindset: routes should coordinate the request and response. Keep connection setup in one place. Avoid opening a brand-new database client inside every route.

PART B — Database foundations before connecting

5. What changes when a database is introduced?

In Day 2, the patient list was written directly inside Python. That data disappears when the process stops. A database stores records outside the running Python process, so the API can retrieve them later.

Layer	Job	Example
Client	Sends an HTTP request	Browser or frontend
FastAPI route	Receives and validates request	POST /patients
Pydantic model	Checks incoming data shape	PatientCreate
Database driver	Communicates with database	PyMongo
Database	Stores records persistently	MongoDB collection
Response model	Controls public output	PatientPublic

6. SQL vs NoSQL — the simple distinction

Topic	MongoDB (NoSQL document database)	PostgreSQL (relational database)
Main storage unit	Document, commonly BSON	Row in a table
Structure	Flexible document fields; schema can be enforced by your app	Tables with defined columns and data types
Relationships	Embedded documents or references	Foreign keys and joins
Query style	MongoDB query operators	SQL
Python learning path	PyMongo driver	SQLModel / SQLAlchemy with a PostgreSQL driver
Good fit examples	Document-shaped data and flexible records	Structured relationships and transactional data

Neither database is automatically better. Choose based on data shape, relationships, consistency requirements, operational needs and team experience. FastAPI does not force you to choose a particular database.

7. Understand the MongoDB words

Term	Meaning	Hospital ERP example
Database	A named container for collections	hospital_erp
Collection	A group of related documents	patients
Document	One stored record	One patient's details
Field	A named value in a document	name, age, department
_id	MongoDB's unique document identifier	ObjectId value
BSON	Binary JSON-like format MongoDB stores	MongoDB document representation

Example document (illustrative):

```
{
    "_id": ObjectId("..."),
    "name": "Ravi Kumar",
    "age": 35,
    "department": "Cardiology"
}
```

MongoDB documents resemble JSON, but MongoDB stores BSON. The _id field is generated automatically when it is not supplied. ObjectId is not a normal string, so convert it to a string before returning it in a JSON API response.

PART C — MongoDB with FastAPI (main practical path)

8. What driver should you use?

Use the official PyMongo package. For this lesson, the examples use PyMongo's asynchronous API with AsyncMongoClient, which fits naturally with async FastAPI endpoints. MongoDB's current documentation describes PyMongo Async as the replacement direction for Motor. Older tutorials may still show Motor; do not mix the two APIs in one example.

Driver note: PyMongo Async is used here. For a small synchronous application, synchronous PyMongo is also valid. If you use AsyncMongoClient, await asynchronous database operations and use async route functions.

9. Set up MongoDB

1. Choose either MongoDB Atlas (managed cloud database) or a local MongoDB Community Server installation.

2. For Atlas, create a database deployment, create a database user, and allow network access only from trusted IP addresses.

3. Copy the application's connection URI from the Atlas connection instructions. Keep the password private.

4. Create a database named hospital_erp and use a collection named patients. MongoDB can create these when the first document is inserted.

Install packages inside your virtual environment:

```
python -m pip install "fastapi[standard]" pymongo pydantic-settings
```

Example .env file:

```
MONGODB_URI=mongodb+srv://<username>:<password>@<cluster-host>/
MONGODB_DATABASE=hospital_erp
```

Replace the placeholders with your own Atlas URI. Do not include angle brackets literally. If your password contains special characters, use the URI-encoding guidance from MongoDB.

.gitignore:

```
.venv/
__pycache__/
.env
*.pyc
```

Never upload .env to GitHub or paste a real database URI into screenshots, public chat, or source control. If a credential is exposed, rotate it.

10. Read configuration safely

```
# app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    mongodb_uri: str
    mongodb_database: str = "hospital_erp"

    model_config = SettingsConfigDict(
        env_file=".env",
        extra="ignore"
    )

settings = Settings()
```

BaseSettings reads configuration from environment variables and the .env file. Keeping secrets outside the Python source makes it easier to use different settings for development and deployment.

11. Create and close the MongoDB client using lifespan

A database client manages connections. Create it once when the application starts, make it available to the application, and close it when the application shuts down. FastAPI's lifespan context is the modern place for this

startup/shutdown work.

```
# app/database.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from pymongo import AsyncMongoClient
from app.config import settings

@asynccontextmanager
async def lifespan(app: FastAPI):
    client = AsyncMongoClient(settings.mongodb_uri)
    app.state.mongo_client = client
    app.state.db = client[settings.mongodb_database]

    # Verify that the server can be reached.
    await client.admin.command("ping")

    try:
        yield
    finally:
        await client.close()
```

The ping checks connectivity during startup. app.state stores the client and selected database for the lifetime of this app process. The finally block closes the client even when shutdown follows an error.

Attach lifespan to the FastAPI app:

```
# app/main.py
from fastapi import FastAPI
from app.database import lifespan
from app.routers import patients

app = FastAPI(
    title="Hospital ERP API",
    lifespan=lifespan
)

app.include_router(patients.router)

@app.get("/")
async def home():
    return {"message": "Hospital ERP API is running"}
```

12. Define the Pydantic models

```
# app/models/patient.py
from pydantic import BaseModel, Field

class PatientCreate(BaseModel):
    name: str = Field(min_length=2, max_length=80)
    age: int = Field(ge=0, le=120)
    department: str = Field(min_length=2, max_length=50)

class PatientUpdate(BaseModel):
    name: str | None = Field(default=None, min_length=2, max_length=80)
    age: int | None = Field(default=None, ge=0, le=120)
    department: str | None = Field(default=None, min_length=2, max_length=50)

class PatientPublic(BaseModel):
    id: str
    name: str
    age: int
    department: str
```

The database uses _id, but the API uses id as a string. PatientPublic is the public response shape. This avoids sending raw BSON ObjectId values.

13. Build patient CRUD endpoints

CRUD means Create, Read, Update and Delete. The following router demonstrates all four operations using MongoDB. Each endpoint receives the database from app.state, validates input with Pydantic, and returns JSON-compatible values.

```
# app/routers/patients.py
from bson import ObjectId
from fastapi import APIRouter, HTTPException, Request, status
from app.models.patient import PatientCreate, PatientUpdate, PatientPublic

router = APIRouter(prefix="/patients", tags=["Patients"])

def to_public(doc: dict) -> dict:
    return {
        "id": str(doc["_id"]),
        "name": doc["name"],
        "age": doc["age"],
        "department": doc["department"],
    }

def get_collection(request: Request):
    return request.app.state.db["patients"]

@router.post("/", response_model=PatientPublic,
             status_code=status.HTTP_201_CREATED)
async def create_patient(patient: PatientCreate, request: Request):
    collection = get_collection(request)
    result = await collection.insert_one(patient.model_dump())
    saved = await collection.find_one({"_id": result.inserted_id})
    return to_public(saved)

@router.get("/", response_model=list[PatientPublic])
async def list_patients(request: Request, limit: int = 20):
    if limit < 1 or limit > 100:
        raise HTTPException(status_code=400, detail="limit must be 1 to 100")
    collection = get_collection(request)
    cursor = collection.find().limit(limit)
    documents = await cursor.to_list(length=limit)
    return [to_public(doc) for doc in documents]

@router.get("/{patient_id}", response_model=PatientPublic)
async def get_patient(patient_id: str, request: Request):
    if not ObjectId.is_valid(patient_id):
        raise HTTPException(status_code=400, detail="Invalid patient ID")
    collection = get_collection(request)
    doc = await collection.find_one({"_id": ObjectId(patient_id)})
    if doc is None:
        raise HTTPException(status_code=404, detail="Patient not found")
    return to_public(doc)

@router.patch("/{patient_id}", response_model=PatientPublic)
```

```
async def update_patient(
    patient_id: str, patient: PatientUpdate, request: Request
):
    if not ObjectId.is_valid(patient_id):
        raise HTTPException(status_code=400, detail="Invalid patient ID")
    changes = patient.model_dump(exclude_unset=True)
    if not changes:
        raise HTTPException(status_code=400, detail="No fields supplied")
    collection = get_collection(request)
    result = await collection.update_one(
        {"_id": ObjectId(patient_id)},
        {"$set": changes}
    )
    if result.matched_count == 0:
        raise HTTPException(status_code=404, detail="Patient not found")
    doc = await collection.find_one({"_id": ObjectId(patient_id)})
    return to_public(doc)

@router.delete("/{patient_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_patient(patient_id: str, request: Request):
    if not ObjectId.is_valid(patient_id):
        raise HTTPException(status_code=400, detail="Invalid patient ID")
    collection = get_collection(request)
    result = await collection.delete_one({"_id": ObjectId(patient_id)})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Patient not found")
    return None
```

This is a teaching example, not a production-ready Hospital ERP. Before real use, add authentication, authorization, audit logging, indexes, backups, privacy controls, pagination strategy and careful error handling.

14. Understand each MongoDB operation

Code	What it does	Plain meaning
insert_one(document)	Creates a document	Register a new patient
find()	Creates a cursor for matching documents	Start reading patient records
find_one(filter)	Returns one matching document or None	Find a patient by ID
update_one(filter, {$set: ...})	Changes selected fields	Update department or age
delete_one(filter)	Deletes one matching document	Remove a patient record
ObjectId.is_valid(value)	Checks whether a string can represent an ObjectId	Reject malformed IDs before querying

15. Run and test the MongoDB application

```
# From the backend-day-04 project folder:
python -m pip install "fastapi[standard]" pymongo pydantic-settings

# Start the API:
fastapi dev app/main.py
```

• Open http://127.0.0.1:8000/ and confirm the home response.

• Open http://127.0.0.1:8000/docs.

• POST /patients/ with name, age and department.

• Copy the returned id and use GET /patients/{patient_id}.

• Use GET /patients/ to list saved records.

• PATCH /patients/{patient_id} with only one field, such as department.

• DELETE /patients/{patient_id}, then GET the same ID again.

• Stop and restart the server. Confirm that the remaining records are still stored.

If data remains after a restart, that is evidence that it was saved in the database rather than only in Python memory.

PART D — PostgreSQL as the second option

16. When PostgreSQL may be a better fit

PostgreSQL is a relational database. It stores data in tables, rows and columns, and supports relationships using primary keys and foreign keys. It is a strong option when your application has structured relationships—for example, patients, doctors, departments, appointments and invoices.

17. Install the relational database tools

```
python -m pip install sqlmodel "psycopg[binary]"
```

This learning path uses SQLModel, which combines SQLAlchemy and Pydantic-style models. Psycopg 3 is the PostgreSQL adapter. Install PostgreSQL locally or create a managed PostgreSQL database, then create a database and a restricted application user.

Example environment variable:

```
DATABASE_URL=postgresql+psycopg://app_user:your_password@localhost:5432/hospital_erp
```

Replace the username, password, host and database with your own values. Keep the URL in .env and do not commit it.

18. A minimal SQLModel connection example

```
# app/sql_database.py
from sqlmodel import SQLModel, create_engine, Session
from app.config import settings

# For this example, add database_url to your Settings class.
engine = create_engine(settings.database_url, echo=True)

def create_db_and_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

Example table model:

```
# app/models/patient_sql.py
from sqlmodel import SQLModel, Field

class Patient(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    age: int
    department: str
```

Example route using a session:

```
# app/routers/patients_sql.py
from typing import Annotated
from fastapi import APIRouter, Depends
from sqlmodel import Session, select
from app.sql_database import get_session
from app.models.patient_sql import Patient

router = APIRouter(prefix="/sql-patients", tags=["PostgreSQL"])

SessionDep = Annotated[Session, Depends(get_session)]

@router.post("/")
def create_patient(patient: Patient, session: SessionDep):
    session.add(patient)
    session.commit()
    session.refresh(patient)
    return patient

@router.get("/")
def list_patients(session: SessionDep):
    return session.exec(select(Patient)).all()
```

For a complete SQLModel application, define separate input/table/output models, create tables during controlled setup, include the router, and use migrations such as Alembic as the schema evolves. The small snippet above is an introductory pattern, not a complete production design.

19. MongoDB vs PostgreSQL — which path are we following?

Decision point	MongoDB	PostgreSQL
Primary lesson path	Yes — main hands-on path in this PDF	Alternative overview and starter pattern
Data model	Documents and collections	Tables and relationships
Python library in this lesson	PyMongo Async	SQLModel + Psycopg 3
ID example	ObjectId	Integer or UUID primary key
Update style	$set document fields	SQL UPDATE / ORM object changes
Relationship example	References or embedded documents	Foreign keys and joins
What to practise first	MongoDB CRUD through FastAPI	Tables, primary keys, foreign keys, sessions and migrations

Recommendation for this learning sequence: complete the MongoDB CRUD lab first so you understand the full API-to-database flow. Then repeat a small create/list exercise with PostgreSQL to understand relational storage.

PART E — Understand the complete request flow

20. Trace a POST /patients request

```
Browser / frontend
      |
      | POST /patients + JSON body
      v
FastAPI application
      |
      v
Patient router
      |
      v
Pydantic PatientCreate validation
      |
      v
Route function
      |
      v
MongoDB collection: patients
      |
      | insert_one(...)
      v
MongoDB stores document and creates _id
      |
      v
Convert ObjectId to string + public response
      |
      v
HTTP 201 Created + JSON response
```

If validation fails, the request is rejected before the database insert. If the ID is malformed, your route can return 400. If the ID is valid but no record exists, return 404. If the database is unreachable during startup, the app should fail clearly rather than pretending it is connected.

21. Common database connection problems

Symptom	Possible cause	What to check
Server fails during startup	Wrong URI, network access or database unavailable	Check URI, Atlas IP access list, credentials and cluster status.
Authentication failed	Wrong database username/password or auth configuration	Recheck the database user; URL-encode special password characters.
DNS / SRV lookup error	Invalid Atlas hostname or DNS issue	Copy the URI again from Atlas; check network/DNS.
No module named pymongo	Package installed in a different Python environment	Activate .venv and run python -m pip show pymongo.
ObjectId serialization error	Raw BSON ObjectId returned to JSON response	Convert _id to a string in a response-mapping function.
GET returns no records	Wrong database or collection, or no records inserted	Check MONGODB_DATABASE, collection name and POST result.
PATCH changes every field unexpectedly	Update model or dump strategy is incorrect	Use exclude_unset=True for partial updates.
Connection works locally but not in deployment	Network policy or environment variables differ	Configure deployment secrets and database network access.

22. Security and reliability rules

• Never hard-code database usernames, passwords or connection strings.

• Use a database user with only the permissions the application needs.

• Do not expose database credentials or raw internal error details to API clients.

• Validate input and authorize the user before reading or changing sensitive records.

• Use TLS for remote database connections and restrict network access.

• Create indexes for fields used frequently in lookups; add unique indexes for identifiers that must be unique.

• Plan backups, restore tests, monitoring and database migrations before production.

• Do not use real patient information in practice projects.

23. Day 4 practical assignment

1. Create the app folder structure shown in this lesson and move patient routes into a router.

2. Create a MongoDB Atlas database or local MongoDB database named hospital_erp.

3. Configure .env and .gitignore correctly.

4. Connect with AsyncMongoClient using the FastAPI lifespan.

5. Implement POST, GET list, GET by ID, PATCH and DELETE for patients.

6. Test valid data, invalid data, malformed ObjectId, missing ID and database-unavailable behaviour.

7. Restart the API and confirm records persist.

8. Draw the request lifecycle from the browser to MongoDB and back.

9. Write a short comparison of MongoDB documents and PostgreSQL tables.

24. Day 4 knowledge check

■ Why should a growing FastAPI application be split into modules?
Answer: _____

■ What does APIRouter do, and what does include_router do?
Answer: _____

■ What is the difference between a database, collection and document?
Answer: _____

■ What is the purpose of a database driver?
Answer: _____

■ Why does this lesson use AsyncMongoClient with async endpoints?
Answer: _____

■ What is the role of FastAPI lifespan?
Answer: _____

■ Why do we convert MongoDB ObjectId to a string?
Answer: _____

■ What is the difference between insert_one, find_one, update_one and delete_one?
Answer: _____

■ Why use model_dump(exclude_unset=True) for PATCH?
Answer: _____

■ How are PostgreSQL tables and foreign keys different from MongoDB documents?
Answer: _____

■ Why must database credentials be stored outside source code?
Answer: _____

Final mini-project — Hospital ERP Patient API

Build a small API that demonstrates everything learned in Days 1–4. Keep the scope limited to patient records. Do not add authentication, appointments or doctor scheduling yet.

Requirement	Expected result
GET /	Health message confirming API process is running
POST /patients/	Validates and saves a patient; returns 201
GET /patients/	Returns a limited list of stored patients
GET /patients/{id}	Returns one patient or a clear 400/404
PATCH /patients/{id}	Updates only fields supplied by the client
DELETE /patients/{id}	Deletes the record or returns 404
/docs	Shows all endpoints grouped under Patients

Overall completion checklist

• I can validate request data with Pydantic Field.

• I can return useful HTTP status codes and error details.

• I can explain why create, update and response models are separate.

• I can organise routes using APIRouter and include_router.

• I understand modules, packages and a basic backend folder structure.

• I can explain the roles of FastAPI, Pydantic, a database driver and a database.

• I can connect FastAPI to MongoDB using PyMongo Async.

• I can perform MongoDB create, read, update and delete operations.

• I can convert ObjectId to a JSON-safe string.

• I understand the basic PostgreSQL + SQLModel alternative.

• I can test success and failure cases in /docs.

• I can explain the request flow without reading the code.

Suggested study schedule

Session	Time	What to do
Day 3 — Concepts	45–60 min	Read sections 1–6 and explain each example aloud.
Day 3 — Lab	60–90 min	Build validation and error-handling endpoints; test in /docs.
Day 3 — Review	20–30 min	Answer knowledge check and complete assignment.
Day 4 — Structure	45–60 min	Create folders, modules and APIRouter.
Day 4 — MongoDB setup	45–60 min	Set up database, environment variables and connection lifespan.
Day 4 — CRUD lab	90–120 min	Build and test all patient CRUD endpoints.
Day 4 — Review	20–30 min	Complete checklist and explain MongoDB vs PostgreSQL.

Official documentation — use these as the source of truth

- FastAPI — Bigger Applications / Multiple Files
https://fastapi.tiangolo.com/tutorial/bigger-applications/
- FastAPI — Handling Errors
https://fastapi.tiangolo.com/tutorial/handling-errors/
- FastAPI — Response Model
https://fastapi.tiangolo.com/tutorial/response-model/
- FastAPI — Request Body
https://fastapi.tiangolo.com/tutorial/body/
- FastAPI — Lifespan Events
https://fastapi.tiangolo.com/advanced/events/
- MongoDB — PyMongo Async migration and usage
https://www.mongodb.com/docs/languages/python/pymongo-driver/current/reference/migration/
- MongoDB — Python driver documentation
https://www.mongodb.com/docs/languages/python/pymongo-driver/current/
- MongoDB — Atlas connection guide
https://www.mongodb.com/docs/atlas/connect-your-application/
- FastAPI — SQL (Relational) Databases
https://fastapi.tiangolo.com/tutorial/sql-databases/
- SQLModel documentation
https://sqlmodel.tiangolo.com/
- Psycopg 3 documentation
https://www.psycopg.org/psycopg3/docs/
- Pydantic — Fields
https://docs.pydantic.dev/latest/concepts/fields/

Your Day 4 success is not just “MongoDB is connected.” It is: “I can explain how a request is validated, routed, stored in a database, retrieved, converted into a safe response, and tested.”